最小生成樹與最短路徑

MST 最小化連通所有頂點的總邊權。最短路徑最小化指定端點之間的路徑成本。兩者的狀態與安全選擇條件不同。

依序研究 **cut 與 MST → relaxation、DAG、Bellman–Ford、Dijkstra → APSP、min-plus、Floyd–Warshall、Johnson**。

前置知識為同組的不相交集合單元（原第21章）與優先佇列操作契約，以及第7章圖、路徑與拓樸排序。原第21章介紹 MAKE-SET、UNION、FIND、rank 與路徑壓縮，並證明精細的反阿克曼成本界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先固定問題，再選演算法

問題與條件	正確性依據
無向圖的最小連通成本	cut property 與安全邊交換。
DAG 的單源最短路徑	拓樸序讓前驅先完成。
含負邊的單源最短路徑	Bellman–Ford 的邊數界與負環檢查。
非負邊的單源最短路徑	Dijkstra 抽出後定案的不變量。
所有有序端點對	DP 狀態或 Johnson 勢能重加權。

每一步都對照距離定義、圖的假設、狀態軌跡與成本模型。負邊與負環的影響須分開判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-01｜要最小化的是連通成本

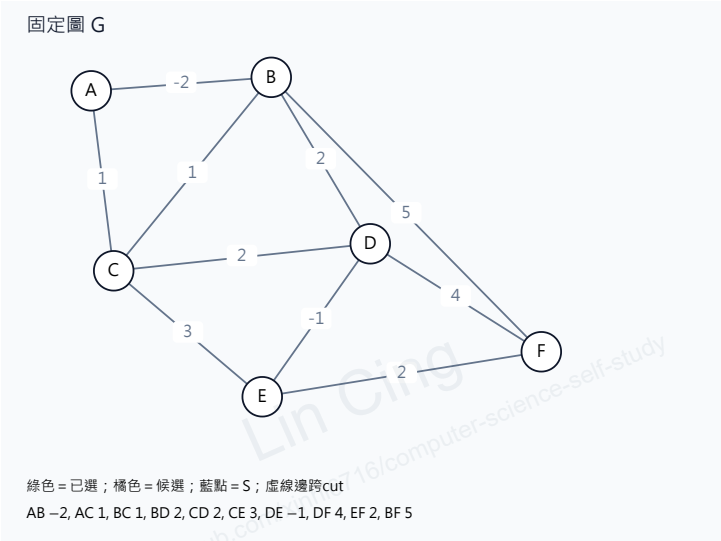
輸入為無向加權圖 G=(V,E)，w:E→ℝ。連通圖的生成樹有 |V|−1 條邊，MST 最小化 Σe∈T w(e)。它不要求每個根到頂點的路徑最短。

前置為同組的不相交集合單元（原第21章）的 MAKE-SET / UNION / FIND、rank 與路徑壓縮，以及 heap 操作契約與第7章圖、路徑、cycle、cut。DSU 的精細攤銷分析見原第21章。

來源 [CLRS23 §23.1; MIT04 L13.8–9; MIT05 L16.8–9]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-02｜同一張圖，兩種貪心



V={A,B,C,D,E,F}。AB=−2，AC=BC=1，BD=CD=EF=2，CE=3，DE=−1，DF=4，BF=5。

A08-MST-03｜安全邊保留至少一個最優解

Kruskal 同權依邊名字典序；Prim 從 A 開始，以 (key,頂點名) 抽最小，只有嚴格變小才換 parent。所有輪次使用這些規則。

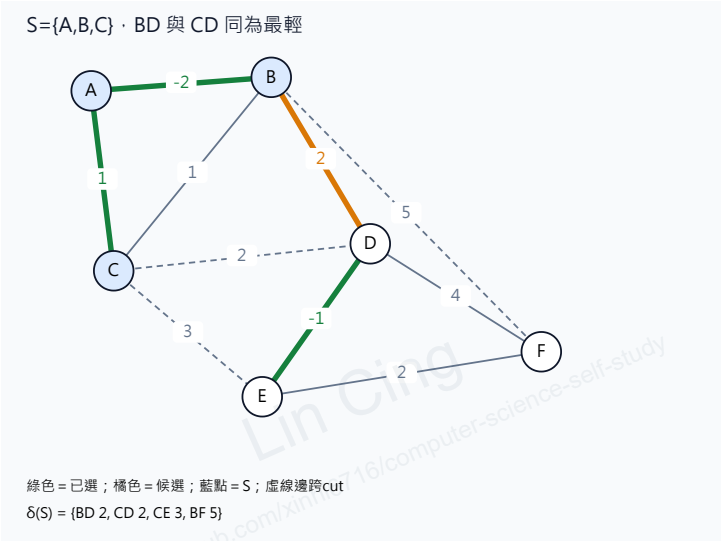
維持不變量 A ⊆ 某棵 MST，並非 A 屬於所有 MST。若 A∪{e} 仍包含於某棵 MST，稱 e 對 A 安全。

cut (S,V−S) 尊重 A，意指 A 沒有邊跨越 cut。跨 cut 的最小權重邊稱 light edge，可能不只一條。

來源 [CLRS23 Theorem 23.1]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-04｜cut property 的 ties 條件



若 A 可延伸成 MST，cut 尊重 A，任一 light edge 都對 A 安全。此例 δ(ABC)={BD 2,CD 2,CE 3,BF 5}，BD 或 CD 都可選。

A08-MST-05｜交換證明的每一步

當 e 是某 cut 唯一最輕邊，e 屬於每棵 MST。有 ties 時只保證可納入某棵 MST，不能推成每棵。

MIT04/05 為簡化假設所有權重相異；本章採 CLRS 的一般條件。

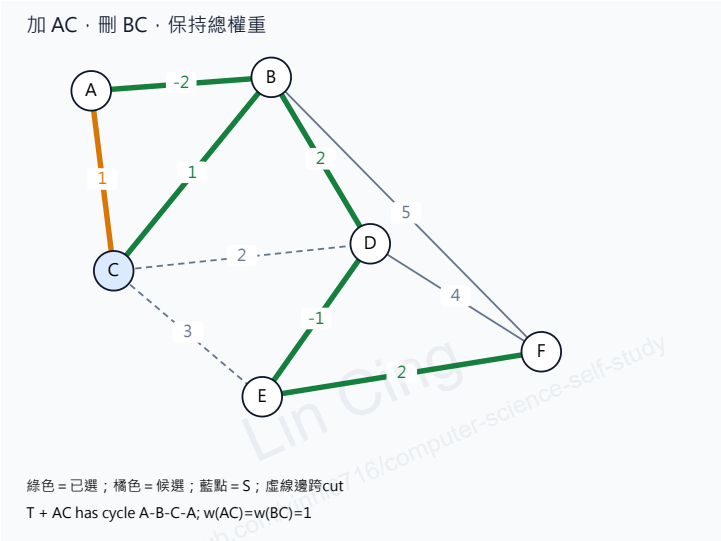
取含 A 的 MST T。若 e∈T 已完成；否則 T+e 出現唯一 cycle。e 跨 cut，因此 T 中連接其端點的路徑至少有另一條跨 cut 邊 f。

cut 尊重 A ⇒ f∉A。e 最輕 ⇒ w(e)≤w(f)。T′=T−f+e 連通且無 cycle，仍含 A∪{e}，且 w(T′)≤w(T)。

T 已最優，故必有 w(T′)=w(T)。等號容許 ties；若 e 唯一最輕而 e∉T，則嚴格變小會矛盾。

來源 [CLRS23 pp627–628; MIT04 L13.22–25; MIT05 L16.22–25]。

A08-MST-06｜圖上交換能保住 A



令 A={AB,DE}，T={AB,BC,BD,DE,EF}。cut S={C} 尊重 A；AC 和 BC 都是最輕邊，權重 1。加 AC 產生 A–B–C–A；刪除不在 A 的 BC。

A08-MST-07｜cycle property 不能省略嚴格二字

得到 {AB,AC,BD,DE,EF}，總權重同為 2。此圖也展示「safe」不代表「原本就在指定 T 裡」。

若 e 是某 cycle 唯一最重邊，e 不屬於任何 MST。證明時假設 T 含 e，刪 e 形成 cut；cycle 的其他邊有跨 cut 的 f，且 w(f)<w(e)，交換就更便宜。

若 e 只是並列最重，只能說存在某棵 MST 不含 e。三角形三邊都為 1 時，每邊都可能被選，也都可能被省略。不可同時刪光並列邊。

來源 [CLRS23 exercise 23.1-5，推導]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-08｜Kruskal 用分量決定能否選

依 (weight,edge-name) 遞增掃邊。FIND(u)≠FIND(v) 才選 uv 並 UNION；相同即有已選路徑，加入會成 cycle。

每次接受邊是連接某個當前分量與外界的 light edge，其分量 cut 尊重 A。DSU 只管當前 A 的連通性，不是原圖 G 的連通性。

來源 [CLRS23 §23.2]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-09｜Kruskal 全部十輪

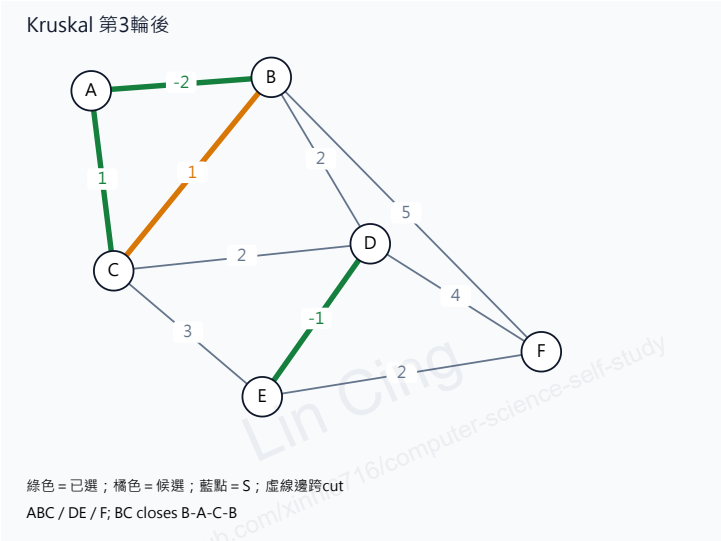
輪次 / Round	邊 / Edge	判定 / Decision	DSU 分量 / Components after round
0	∅	初始化 / initialize	A / B / C / D / E / F
1	AB, −2	選 / accept	AB / C / D / E / F
2	DE, −1	選 / accept	AB / C / DE / F
3	AC, 1	選 / accept	ABC / DE / F
4	BC, 1	拒 / reject, B–A–C 已連通 / connected	ABC / DE / F
5	BD, 2	選 / accept	ABCDE / F

A08-MST-09B｜Kruskal 後五輪

輪次 / Round	邊 / Edge	判定 / Decision	DSU 分量 / Components after round
6	CD, 2	拒 / reject, C–A–B–D 已連通 / connected	ABCDE / F
7	EF, 2	選 / accept	ABCDEF
8	CE, 3	拒 / reject, C–A–B–D–E 已連通 / connected	ABCDEF
9	DF, 4	拒 / reject, D–E–F 已連通 / connected	ABCDEF
10	BF, 5	拒 / reject, B–D–E–F 已連通 / connected	ABCDEF

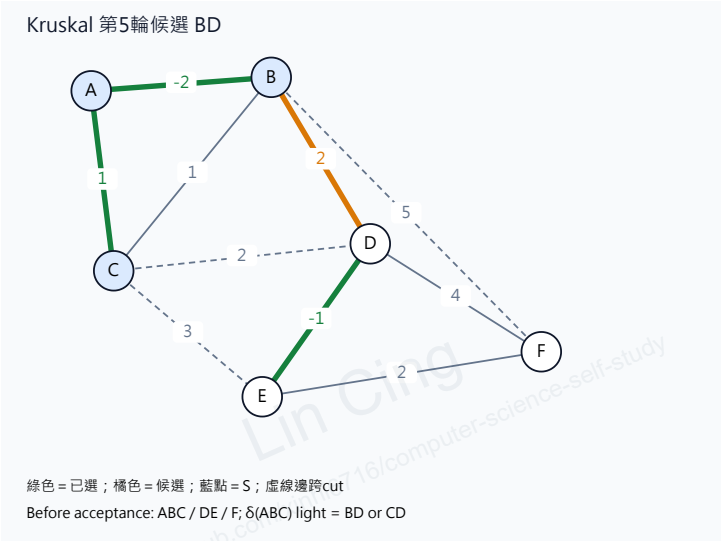
第7輪已取得5條邊可提早停止。表格仍掃完全部10邊，讓每個拒邊原因可核對。DSU欄列分量，不聲稱列出內部parent樹。

A08-MST-10｜拒 BC，接 BD



第4輪 BC 不是因權重大才拒絕，因 B、C 已同分量。

A08-MST-10｜拒 BC，接 BD （續）



第5輪 BD 的 cut 證據正是卡04。CD 同權可代替 BD，但排在 BD 後，之後已成環。

A08-MST-11｜Prim 的 key 是一條邊的成本

S 是已抽出頂點。對 v∉S，key[v]=min{w(u,v):u∈S}，不存在則為 ∞；parent[v] 記錄達到最小值的 u。

抽 key 最小的 v，parent 邊是跨 (S,V−S) 的 light edge。只在 v 還在 Q 且 w(u,v)<key[v] 時更新。初始化 key[A]=0 僅為選根，並非實際邊。

來源 [CLRS23 pp634–636; MIT04 L13.26; MIT05 L16.26]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

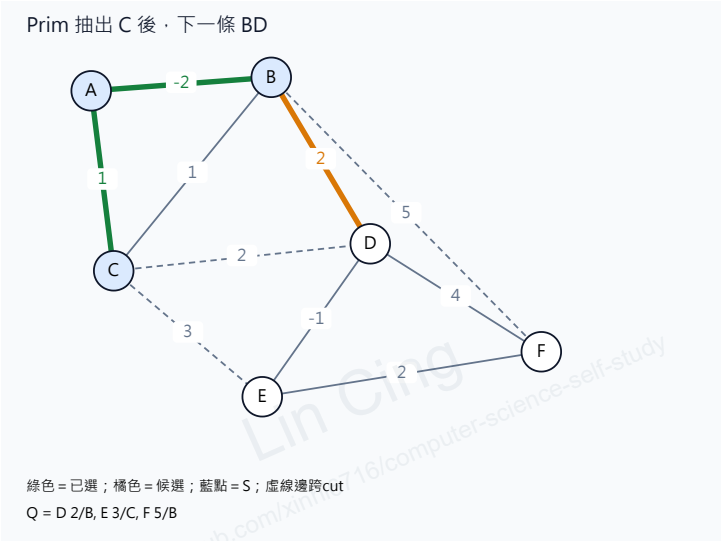
A08-MST-12｜Prim 全部 key、parent 與 PQ

抽出 / Extract	抽出前 S / S before	加入 / Add	掃鄰邊後 Q，key/parent，優先序 / Q after relaxation, in priority order
初始 / Initial	∅	∅	A 0/∅, B ∞/∅, C ∞/∅, D ∞/∅, E ∞/∅, F ∞/∅
A	∅	根 / root	B −2/A, C 1/A, D ∞/∅, E ∞/∅, F ∞/∅
B	A	AB	C 1/A, D 2/B, F 5/B, E ∞/∅
C	AB	AC	D 2/B, E 3/C, F 5/B
D	ABC	BD	E −1/D, F 4/D
E	ABCD	DE	F 2/E
F	ABCDE	EF	∅

Q列出邏輯優先序，不是binary heap的陣列位置。未出現在Q的頂點已在S，最終parent為B←A,C←A,D←B,E←D,F←E。

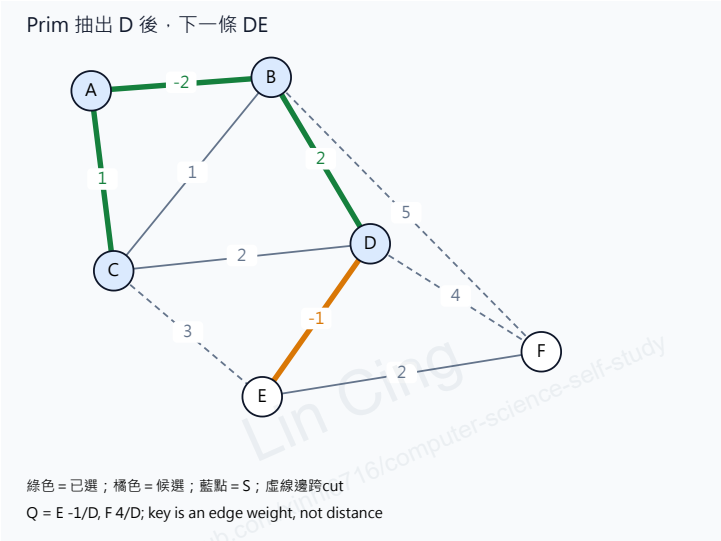
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-13｜每次抽出都有一個 cut



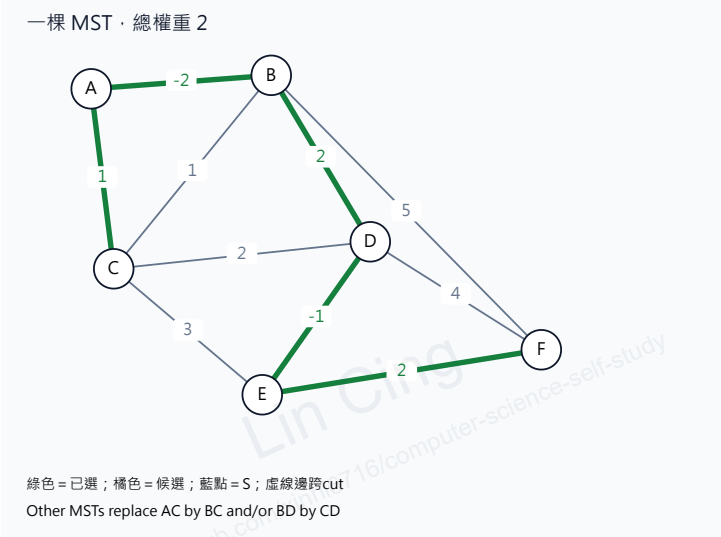
C抽出後，S=ABC，Q={D,E,F}；δ(S) 為 BD 2,CD 2,CE 3,BF 5。Q中D的key=2，parent=B。CD=2沒有嚴格更小，因此保留B。

A08-MST-14｜負權不破壞 MST 貪心



D抽出前key[D]=2；掃D後key[E]=−1。抽出key序列0,−2,1,2,−1,2並不單調。

A08-MST-15｜共同答案與非唯一



T={AB,AC,BD,DE,EF}，w(T)=−2+1+2−1+2=2。獨立枚舉全部 C(10,5)=252 個五邊子集，篩連通樹後確認最小值2，共4棵MST。

A08-MST-15A｜跨 cut、負權與最優解

其餘抽出前cut依序為 S={A}、AB、ABCD、ABCDE，最輕候選分別 AB −2、AC/BC 1、DE −1、EF 2。根A沒有parent邊。

交換證明只比較邊權，不要求非負。Prim不做 key[u]+w(u,v)，因此不是Dijkstra的距離鬆弛。

全部最優集合為 {AB,DE,EF}∪{AC或BC}∪{BD或CD}。權重相異足以保證唯一；有重複權重不必然不唯一。

A08-MST-16｜不連通時交付 minimum spanning forest

若圖有 c 個連通分量，跨所有頂點的生成樹不存在。分量各自的 MST 合成 MSF，邊數 |V|−c。

固定圖若只保留 {AB −2,AC 1,BC 1,DE −1,DF 4,EF 2}，分成 ABC 與 DEF。MSF={AB,AC,DE,EF}，權重0，另一棵以BC換AC。

Kruskal 掃完即得 MSF。Prim 當剩餘最小key=∞，以該頂點為新根，parent=∅，不加入∞假邊，再繼續掃鄰邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-17｜成本跟實作一起寫

令 n=|V|、m=|E|。Kruskal比較排序 O(m log m)，DSU初始化與操作 O((n+m)(1+α(n)))，沿用原第21章的界。簡單連通圖可合寫 O(m log n)。

Prim 實作	抽最小	降低key	整體時間
陣列掃描＋鄰接矩陣	O(n)	O(1)	$O(n^2)$
indexed binary heap＋鄰接串列	O(log n)	O(log n)	O((n+m) log n)
Fibonacci heap＋鄰接串列	O(log n) 攤銷	O(1) 攤銷	O(m+n log n)

indexed heap需維護頂點位置，Q成員測試O(1)。lazy heap每次改善push新紀錄，pop時丟棄過期紀錄，可能有O(m)個項目，界為O(n+m log(m+1))、額外O(n+m)空間；不能宣稱其heap只有n項。

來源 [CLRS23 pp633,636; MIT04 L13.40–50; MIT05 L16.40–50]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A08-MST-18｜把證明、狀態、答案互相核對

選邊先指出尊重A的cut與light條件；拒邊先給已選路徑。Prim每輪讀 key/parent/Q，Kruskal每輪讀 DSU分量。ties只決定哪棵最優樹，負權不改安全性。

練習 A08-M01–08 分別檢查cut、交換、cycle、DSU、PQ、MSF、唯一性與成本。`check.py`獨立枚舉所有生成樹並驗證這兩張完整追蹤表。

原筆記PDF pp36–37提供交換思路與第三輕邊反例；本章用CLRS23補足cut尊重A與ties的精確敘述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單源最短路徑的三種答案

給有限有向圖、有限實數邊權與來源 s。定義 δ(s,v) 為所有 s→v walk 權重的下確界。不可達為 ∞；可沿途繞可達負環再到 v 為 −∞；其餘可達點存在有限最短路。空路使 δ(s,s)≤0。

本節以同一圖追蹤 d、π、佇列，分清演算法「何時可承諾答案」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

負邊與負環的差別

若到 v 的 walk 不能經過負環，刪去正環使成本降低，刪去零環不改成本，因此存在至多 $|V|-1$ 邊的簡單最短路。

全圖無負環是充分條件。對單一 v 的精確條件是沒有「s 可達、且可再到 v」的負環。別支的負環未必破壞 v；不可達負環不影響來源 s。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共同核心是 relaxation

```
initialize: d[s]=0; d[v≠s]=∞; π[v]=NIL
RELAX(u,v,w):
  if d[u] ≠ ∞ and d[v] > d[u]+w:
    d[v]=d[u]+w; π[v]=u
```

∞ 以獨立標記表示。對有限 w，∞+w=∞。整數程式先判斷可達，再用足夠寬的數值加法。不可拿最大整數直接相加，也不能任意向下截斷負值。嚴格改善才改 π；平行邊時 π 應記錄邊 ID。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

relaxation 不變量與收斂

每個有限 d[v] 都是某條實際 s→v walk 的成本。因此 d[v]≥δ(s,v)，估計值只降不升，不可達點永遠為 ∞。

若最短路 s⇝u→v 的前綴已收斂，relax(u,v) 後 v 也收斂。依路徑順序 relax 每一條邊即可傳遞正確值；其間穿插其他 relax 不妨礙。到達有限 δ 後不再改變。

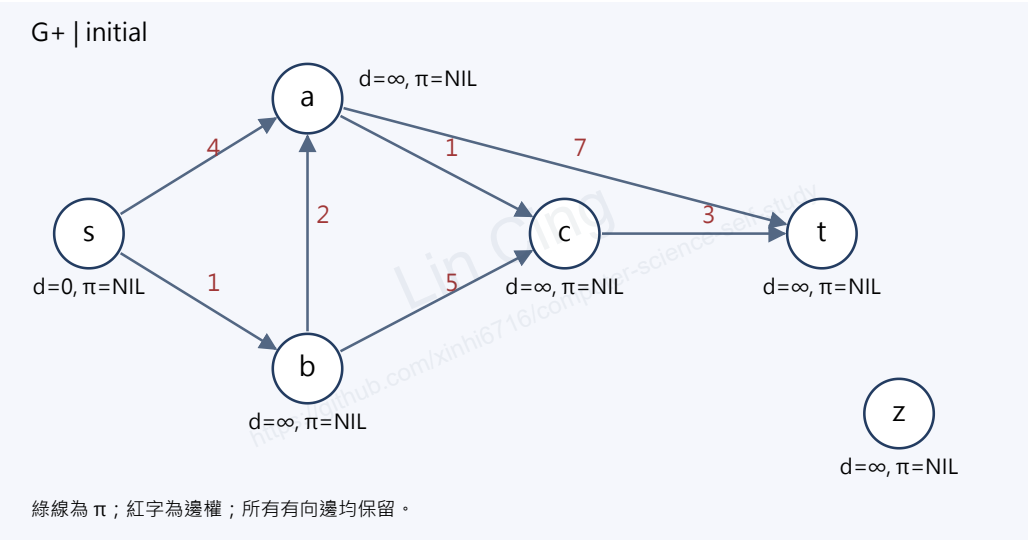
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

固定主圖 G+ 與初始化

頂點順序固定 (s,a,b,c,t,z)，z 孤立。全部邊如下。

s→a:4, s→b:1, b→a:2, a→c:1, b→c:5, c→t:3, a→t:7。

初始化 d=(0,∞,∞,∞,∞,∞)，π 全 NIL。邊及數值表均為自製例，並非教材圖的複製。



DAG 允許負邊

令 G− 與 G+ 相同，僅 a→c 改成 −2。它仍為 DAG。固定拓序 s,b,a,c,t,z。每個 u 被處理前，其所有前驅已完成，所以 d[u] 已正確；無須比較當前最小鍵。

拓序加掃邊為 Θ(V+E)。拓序需先驗證，不能把有環圖任意排一排就套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DAG 的單次傳遞

以下每列完成該點出邊。π 仍按 (s,a,b,c,t,z) 排列。

u	d	π
初始	(0,∞,∞,∞,∞,∞)	(NIL,NIL,NIL,NIL,NIL,NIL)
s	(0,4,1,∞,∞,∞)	(NIL,s,s,NIL,NIL,NIL)
b	(0,3,1,6,∞,∞)	(NIL,b,s,b,NIL,NIL)
a	(0,3,1,1,10,∞)	(NIL,b,s,a,a,NIL)
c	(0,3,1,1,4,∞)	(NIL,b,s,a,c,NIL)
t,z	(0,3,1,1,4,∞)	(NIL,b,s,a,c,NIL)

注意 a 的 3 大於 c 最終的 1，卻必須先處理 a。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman-Ford 固定掃邊順序

在 G− 不使用拓序，固定每 pass 掃描以下七邊，直接寫回 d。

c→t:3, a→t:7, a→c:−2, b→c:5, b→a:2, s→b:1, s→a:4。

執行 |V|−1=5 passes，再檢查有沒有可改善的邊。若某整個 pass 無更新，可提前停。順序影響中途值與收斂輪數，但不影響無可達負環時的最終答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman-Ford 逐 pass 的 d、π

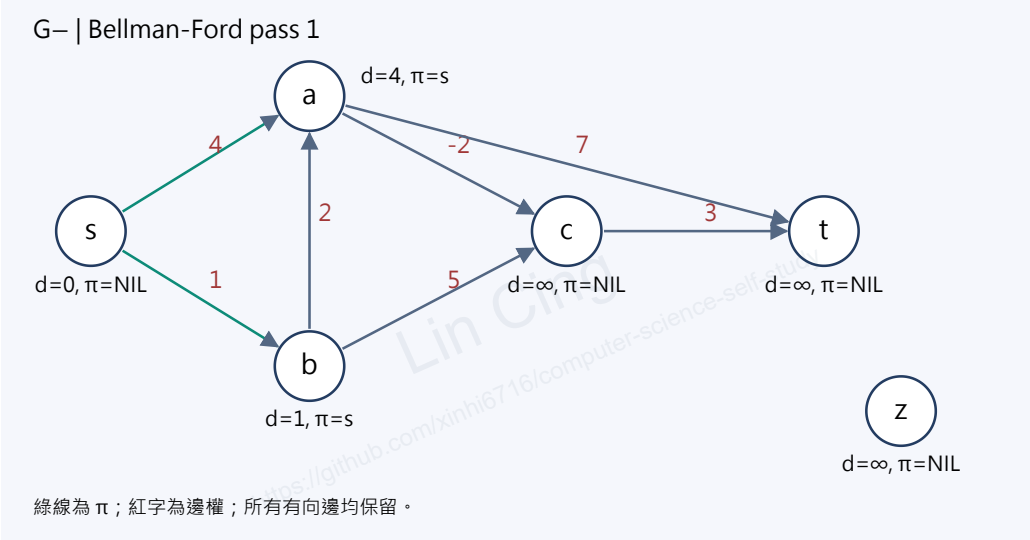
pass	d(s,a,b,c,t,z)	π(s,a,b,c,t,z)
0	(0, ∞, ∞, ∞, ∞, ∞)	(NIL, NIL, NIL, NIL, NIL, NIL)
1	(0, 4, 1, ∞, ∞, ∞)	(NIL, s, s, NIL, NIL, NIL)
2	(0, 3, 1, 2, 11, ∞)	(NIL, b, s, a, a, NIL)
3	(0, 3, 1, 1, 5, ∞)	(NIL, b, s, a, c, NIL)
4	(0, 3, 1, 1, 4, ∞)	(NIL, b, s, a, c, NIL)
5	(0, 3, 1, 1, 4, ∞)	(NIL, b, s, a, c, NIL)

第 1 pass 的尾端才處理 s 出邊。第 2 pass 先用 a=4 令 c=2、t=11，尾端才令 a=3。第 3 pass 先令 t=5 再令 c=1；第 4 pass 才令 t=4。

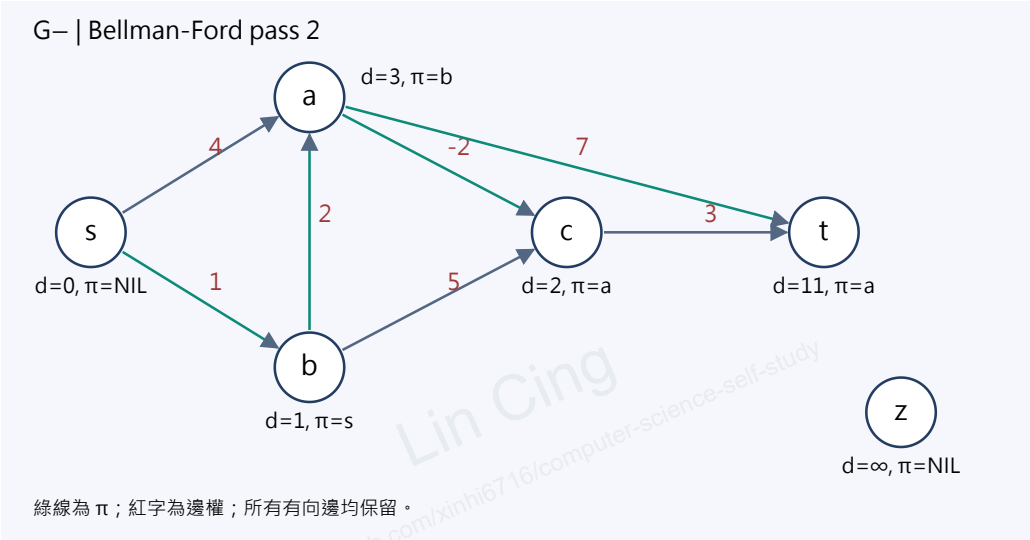
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐 pass 圖必須保留邊與暫存值

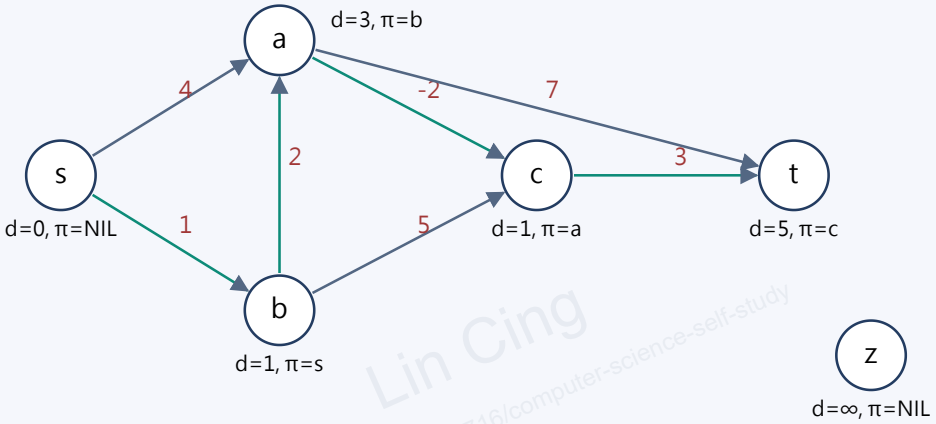
圖與上一表是同一個 G−、同一順序、同一取樣點。第 2 pass 的 π 雖已形成 s→b→a→c，d[c]=2 仍不是目前父鏈的成本 1。父鏈與 d 可能短暫不同步。



逐 pass 圖必須保留邊與暫存值 （續）



逐 pass 圖必須保留邊與暫存值 （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman-Ford 為何只需 V−1 passes

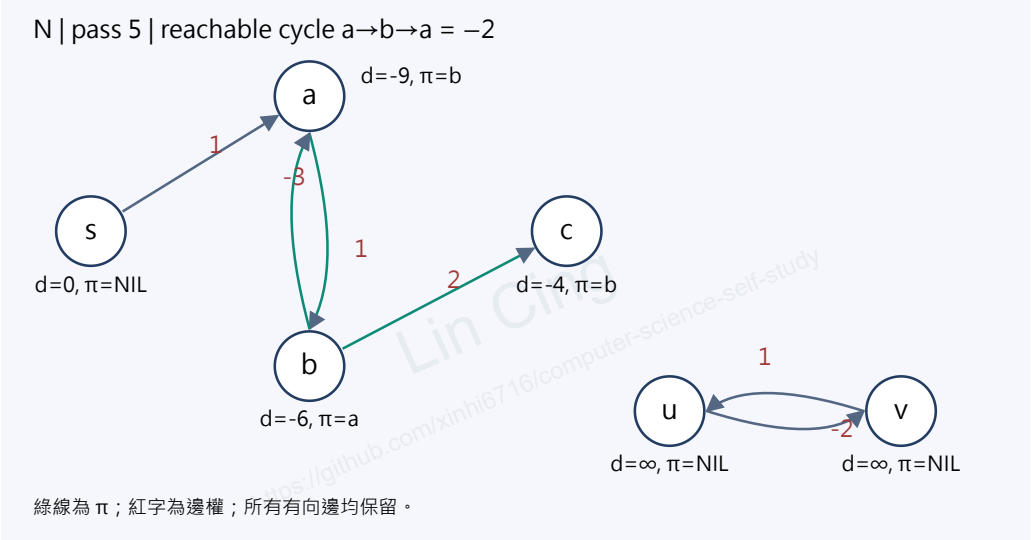
第 k 個完整 pass 後，每個可用 ≤k 邊的最短路已收斂。理由是第 k pass 一定掃到該路的第 k 邊，而其前綴至遲前一 pass 已收斂。

in-place 實作可能一輪傳過多邊，故不可宣稱 d「恰等於限制 k 邊的最短距離」。無可達負環時存在 ≤V−1 邊的簡單最短路，因此結果正確。時間 O(VE)，含初始化寫成 O(V+VE) 可涵蓋 E=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可達負環與受影響後繼

圖 N 的全部邊為 s→a:1,a→b:1,b→a:−3,b→c:2,u→v:−2,v→u:1。u,v 不可達。固定此順序，頂點 (s,a,b,c,u,v)。



負環檢查與受影響頂點

pass 0 到 5 的 d 依次為 (0,∞,∞,∞,∞,∞)、(0,−1,2,4,∞,∞)、(0,−3,0,2,∞,∞)、(0,−5,−2,0,∞,∞)、(0,−7,−4,−2,∞,∞)、(0,−9,−6,−4,∞,∞)。

第 6 次只讀檢查發現 a→b 可改善，因 −9+1<−6。從所有這類目的點做 DFS/BFS，標記 a,b,c 為 −∞；s 保持 0，u,v 為 ∞。不能把全圖全標成 −∞。負環檢測正確性來自沿環把「不可改善」不等式相加會得到 0≤負環權重的矛盾。

Dijkstra 的演算法合約

所有邊權必須非負。S 為已確定頂點，Q=V−S。每輪 extract-min u，加入 S，依鄰接串列 relax 所有 u 的出邊並 decrease-key。

本例 Q 含所有頂點，鍵同值以頂點名稱排序。表中 Q 已排序顯示邏輯內容，不表示 heap 陣列。若最小鍵為 ∞，可提前停止，剩餘點全不可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 每輪完整 d、π、Q

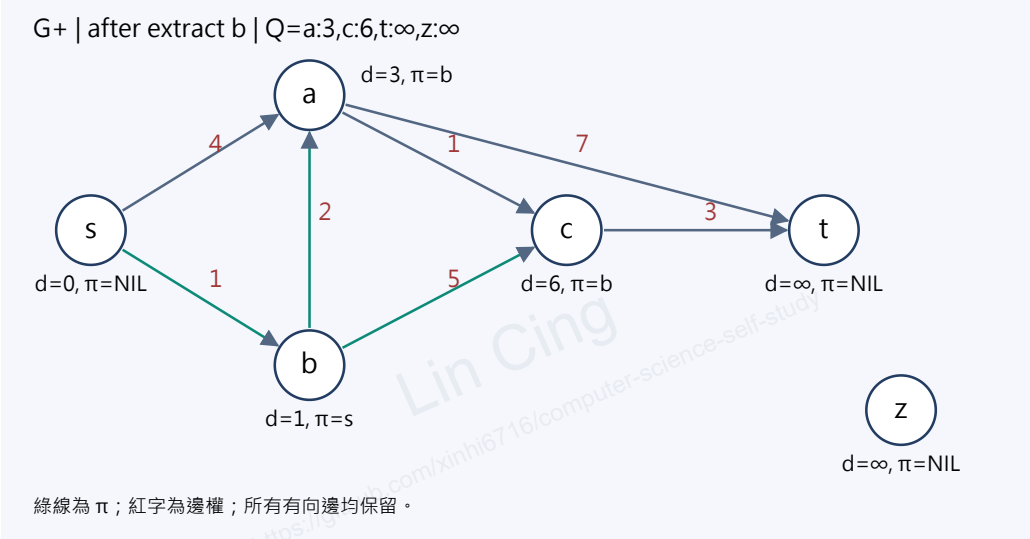
extract	d(s,a,b,c,t,z)	π(s,a,b,c,t,z)	Q after relaxation
init	(0, ∞, ∞, ∞, ∞, ∞)	(NIL, NIL, NIL, NIL, NIL, NIL)	s:0, a:∞, b:∞, c:∞, t:∞, z:∞
s	(0, 4, 1, ∞, ∞, ∞)	(NIL, s, s, NIL, NIL, NIL)	b:1, a:4, c:∞, t:∞, z:∞
b	(0, 3, 1, 6, ∞, ∞)	(NIL, b, s, b, NIL, NIL)	a:3, c:6, t:∞, z:∞
a	(0, 3, 1, 4, 10, ∞)	(NIL, b, s, a, a, NIL)	c:4, t:10, z:∞
c	(0, 3, 1, 4, 7, ∞)	(NIL, b, s, a, c, NIL)	t:7, z:∞
t	(0, 3, 1, 4, 7, ∞)	(NIL, b, s, a, c, NIL)	z:∞
z	(0, 3, 1, 4, 7, ∞)	(NIL, b, s, a, c, NIL)	

每列在該輪出邊全部 relax 後取樣。初始化到 extract s，先得到 a=4,b=1。extract b 才把 a 改成 3，並首次令 c=6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

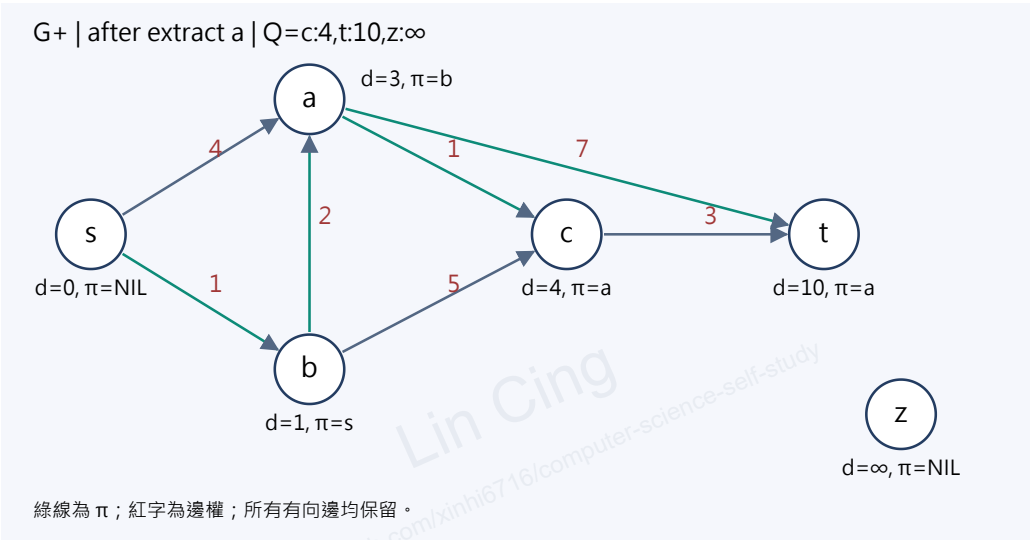
兩次關鍵更新與可見佇列

extract b 後 a=3、c=6；extract a 後 c=4、t=10。因此 c 的父點從 b 改為 a，t 首次由 a 接入。



綠線為 π；紅字為邊權；所有有向邊均保留。

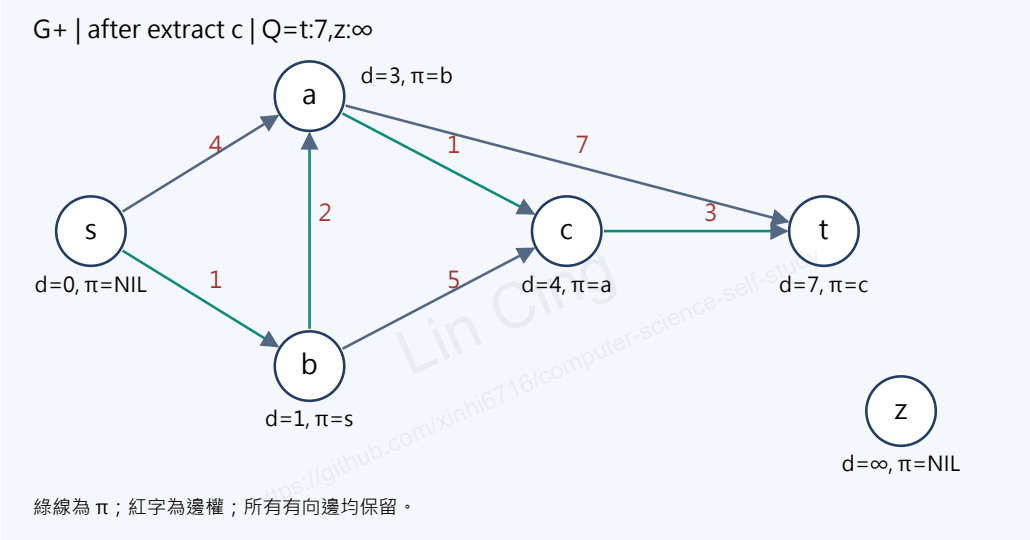
兩次關鍵更新與可見佇列 （續）



綠線為 π；紅字為邊權；所有有向邊均保留。

最後改善與不可達點

extract c 後 t 從 10 降為 7，π[t] 從 a 改為 c。extract t 不改值；最後 z 的 ∞ 不是尚未算出的有限答案。



最短路 s→b→a→c→t 成本 1+2+1+3=7。

為何 extract-min 可確定答案

不變量為每個 x∈S 都有 d[x]=δ(s,x)。設下一個 u 可達，取一條 s→u 最短路，y 為第一個不在 S 的點，x 為其前驅。x 出邊早已 relax，故 d[y]=δ(s,y)。

非負的 y⇝u 後綴給 δ(s,y)≤δ(s,u)。由 extract-min 與上界性，
δ(s,u)≤d[u]≤d[y]=δ(s,y)≤δ(s,u)，全部相等。不可達 u 用 no-path 性質處理。

負邊如何打破 settled 證明

反例 H 為 s→a:2, s→b:5, b→a:−4, a→t:2。抽取順序 s,a,t,b，抽 a 時得到 2，實際 $\delta(s,a)=1$；t 被定為 4，實際為 3。

若禁止修改 S，a 也錯；若允許改 a 卻不重新處理其出邊，t 仍錯。可重開點的修正版不再具這個 Dijkstra 的 settled 證明與成本。失效處正是 $\delta(s,b)\le\delta(s,a)$ 不成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 的優先佇列成本

|V| 次 extract-min，至多 |E| 次成功 decrease-key，每邊掃描一次。

Q 實作	總時間
無序陣列	$O(V^2+E)$
二元 heap	$O((V+E)\log V)$
Fibonacci heap	$O(E+V\log V)$ 攤銷

lazy heap 每改善插入新紀錄、抽出時跳過過期值，可用 $O((V+E)\log(V+E))$ 時間與 $O(V+E)$ 空間；不可把它寫成 decrease-key 版本而沿用 $O(V)$ heap 空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

路徑重建與選法

僅對有限且已確認的距離重建。從目標沿 π 回追到 s，再反轉；若目標 d=∞ 回傳不可達，標為 −∞ 則回傳無有限最短路。加上最多 |V| 步的防護可揭露損壞或尚未定案的父鏈。

G− 最終 t←c←a←b←s，反轉得 s→b→a→c→t，成本 1+2−2+3=4。來源 s 的答案是空路。

圖條件	選法
DAG，可有負邊	拓序 relaxation
一般圖，全非負	Dijkstra
一般圖，可能負邊	Bellman-Ford 與可達負環檢查

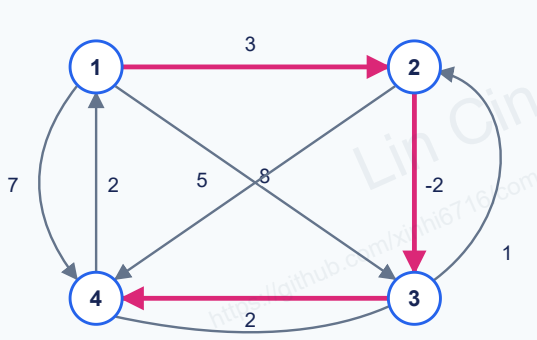
資料來源為原筆記 p42、CLRS 第24章與 MIT04 L14/L15、MIT05 L17/L18；頁碼映射見 sources.json。

APSP 要交付每一對的距離與路徑

輸入有向圖 G=(V,E)，n=|V|，m=|E|。邊可為負。距離矩陣每列是一個來源。無路徑為 +∞；可穿越負環的點對為 −∞，沒有有限最短路徑。

先在無負環條件推導有限距離。W 的直接邊取最小權，無邊為∞，對角線取 min(0,自環權重)，不能抹掉負自環。

四點共用圖 G



8 條邊；唯一負邊 2→3

2→3→4→2 = 1

1→2→3→4 = 3

1→4 = 7

3 < 7

粉紅邊標示本次重點路徑或負環；箭頭表示方向。

本段數例為自製四點圖，並非原書圖。來源 [S1 §25.1–25.3]、[S2 p4–23]、[S3 p4–23]。

邊數 DP 的狀態是「最多」m 條邊

$L^{(t)}[i,j]$ 是最多 t 條邊的 walk 最小權重。$L^{(0)}=I$，I 對角為0，其餘∞。

$$L^{(t+1)}[i,j] = \min_k \{L^{(t)}[i,k] + W[k,j]\}.$$

沿最後一條邊切開得到候選；每個候選也能接成可行 walk。W[j,j] 取 0 與自環權重的最小值，故 W[j,j]≤0。對最多 t 邊的 walk，接上此對角項：若為 0 就接空路，否則接負自環。結果最多 t+1 邊且成本不增加，故即使有負環也涵蓋較短的 walk；固定預算不一定等於無限制距離。

無負環時，刪掉非負圈不增加成本，最短路可取簡單路，最多 n−1 邊。這是停止在 n−1 的依據。[S1 p687–688；S2 p6–9；S3 p6–9]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Min-plus 乘法把邊數預算相加

定義 $(A\otimes B)[i,j]=\min_k(A[i,k]+B[k,j])$。若 $A=L^{(a)}$、$B=L^{(b)}$，則 $A\otimes B=L^{(a+b)}$。任一最多 a+b 邊的 walk 可在第 min(a,實際邊數) 條邊後切開；反向可直接串接。

結合律來自同一個最小值。

$$(A\otimes B)\otimes C = A\otimes(B\otimes C), \qquad \min_{p,q}(A[i,p]+B[p,q]+C[q,j]).$$

I 是單位矩陣。這裡只用有限數與 +∞，避免 ∞+(−∞) 未定義。普通矩陣乘法的「加總、相乘」換成「取最小、相加」；不能直接沿用 Strassen 的減法。[S1 p688–691；S2 p10–14；S3 p10–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Slow 與 faster 改的是計算層數

```
Extend(A,B):
  C = new n×n matrix filled with ∞
  for i,j,k: C[i,j] = min(C[i,j], A[i,k]+B[k,j])
  return C
Slow(W):
  L = I
  repeat n−1 times: L = Extend(L,W)
  return L
Faster(W):
  L = W; t = 1
  while t < n−1: L = Extend(L,L); t = 2*t
  return L
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Min-plus 成本與負環偵測

一次 Extend 為 $\Theta(n^3)$。Slow 共 n−1 次，$\Theta(n^4)$；faster 的預算 1,2,4,…，$\Theta(n^3 \log n)$。只保存輸入與新矩陣即可用 $\Theta(n^2)$ 工作空間，不能邊讀邊覆寫同一乘法的輸入。

上述答案假設無負環。若要靠 min-plus 對角線偵測負環，預算須至少 n，因負的簡單圈可能有 n 條邊。n−1 的停止條件只用於求無負環距離。[S1 p689–692；S2 p13–14；S3 p13–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同圖核算 $L^{(2)}$ 與 $L^{(3)}$

$L^{(1)}=W$。$L^{(2)}$ 把每一對可能的最後一個中繼點全部試過。

i \ j	1	2	3	4
1	0	3	1	7
2	7	0	-2	0
3	4	3	0	2
4	2	1	-1	0

$L^{(3)}$ 已是最短距離，因 n−1=3。

i \ j	1	2	3	4
1	0	3	1	3
2	2	0	-2	0
3	4	3	0	2
4	2	1	-1	0

$L^{(2)}[1,4]=\min(7,3+5,8+2)=7$；$L^{(3)}[1,4]=1+2=3$，路徑1→2→3→4。Faster 算 $L^{(2)}$、$L^{(4)}$，後者等於 $L^{(3)}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floyd–Warshall 改用允許中繼集合

$D^{(k)}[i,j]$ 的所有中繼點屬於 $S_k=\{1,\cdots,k\}$，端點 i,j 不受此限制。$D^{(0)}=W$。

$$D^{(k)}[i,j]=\min\{D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j]\}.$$

無負環時可取簡單最短路。若路徑不經 k，用第一項；若經 k，就在唯一出現的 k 切開，兩段的中繼點都在 S_{k-1}。每個候選串接後也是允許的 walk，刪掉非負圈仍可行。兩方向證明轉移等式。

原筆記用「最短路的子路徑仍最短」引出最優子結構；這裡再加上允許集合，才得到正確可執行的 DP。[S0 p18–19；S1 p693–695；S2 p15–17；S3 p15–17]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

k=0，尚未允許任何中繼點

$D^{(0)}$ 只有直接邊及空路徑。Π[i,j] 存 j 的前一點，並非中繼點編號。i=j 或無路徑為 NIL；直接邊的前驅為 i。

$D^{(0)}$ i \ j	1	2	3	4
1	0	3	8	7
2	∞	0	-2	5
3	∞	∞	0	2
4	2	1	∞	0

$\Pi^{(0)}$ i \ j	1	2	3	4
1	NIL	1	1	1
2	NIL	NIL	2	2
3	NIL	NIL	NIL	3
4	4	4	NIL	NIL

k=1，只開放中繼點 1

唯一改進為 4→3。原本∞，現在 4→1→3 的權為 2+8=10。$\Pi^{(1)}[4,3]=\Pi^{(0)}[1,3]=1$。端點3不必在{1}。

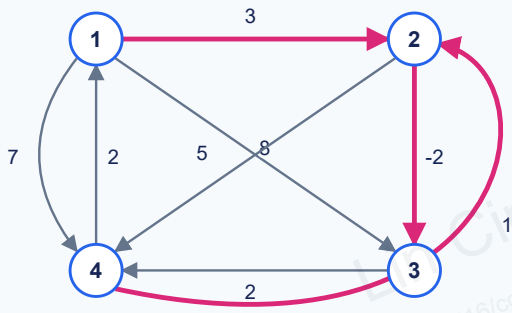
$D^{(1)}$ i \ j	1	2	3	4
1	0	3	8	7
2	∞	0	-2	5
3	∞	∞	0	2
4	2	1	10	0

$\Pi^{(1)}$ i \ j	1	2	3	4
1	NIL	1	1	1
2	NIL	NIL	2	2
3	NIL	NIL	NIL	3
4	4	4	1	NIL

k=2，透過負邊改進兩對

1→3 由8降為1，4→3 由10降為−1。兩者最後一段均為2→3，所以兩個前驅都改成2。

允許中繼點 2 的改進



$S_2 = \{1, 2\}$

1→3: 8 → 3 + (−2) = 1

4→3: 10 → 1 + (−2) = −1

3 是終點，可不在 S_2

1→4 仍為 7

粉紅邊標示本次重點路徑或負環；箭頭表示方向。

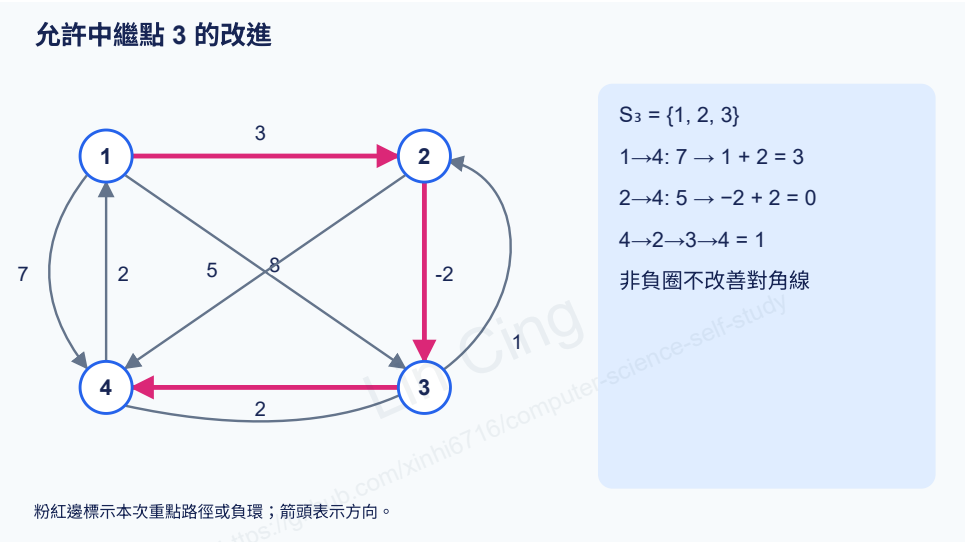
k=2，透過負邊改進兩對 (續)

$D^{(2)}$ i \ j	1	2	3	4
1	0	3	1	7
2	∞	0	-2	5
3	∞	∞	0	2
4	2	1	-1	0

$\Pi^{(2)}$ i \ j	1	2	3	4
1	NIL	1	2	1
2	NIL	NIL	2	2
3	NIL	NIL	NIL	3
4	4	4	2	NIL

k=3，三邊路徑第一次出現

1→4 由7降為3；2→4 由5降為0。Π[1,4]=Π[3,4]=3，不能一律填入 k 當作一般規則。



k=3，三邊路徑第一次出現 (續)

D⁽³⁾ i \ j	1	2	3	4
1	0	3	1	3
2	∞	0	-2	0
3	∞	∞	0	2
4	2	1	-1	0

Π⁽³⁾ i \ j	1	2	3	4
1	NIL	1	2	3
2	NIL	NIL	2	3
3	NIL	NIL	NIL	3
4	4	4	2	NIL

k=4，完成距離並重建路徑

新增 2→1=2、3→1=4、3→2=3，其餘不變。對來源1，從終點4反向查 Π[1,4]=3、Π[1,3]=2、Π[1,2]=1，再反轉得1→2→3→4，權3。

嚴格改善才更新 Π[i,j]=Π[k,j]，相等保留舊路。先檢查可達與負環影響，才重建；零權圈下相等也更新可能抹掉有效前驅。[S1 p695–697]

$D^{(4)}$ i \ j	1	2	3	4
1	0	3	1	3
2	2	0	-2	0
3	4	3	0	2
4	2	1	-1	0

$\Pi^{(4)}$ i \ j	1	2	3	4
1	NIL	1	2	3
2	4	NIL	2	3
3	4	4	NIL	3
4	4	4	2	NIL

k 必須在外圈，才能維持 DP 層

```
D = W
for k = 1..n:
  for i = 1..n:
    for j = 1..n:
      if D[i,k]+D[k,j] < D[i,j]:
        D[i,j] = D[i,k]+D[k,j]
        Π[i,j] = Π[k,j]
```

外圈 k 完成後，每一對都已納入 S_k。無負環時 D[k,k]=0，所以本層的第 k 列與第 k 欄不變；原地讀取仍是前一層所需值。i、j 可互換，任意把 k 移到內圈則無此保證。

反例只有 1→4=1、4→3=1、3→2=1。依遞增 i,j,k 只跑一遍，算(1,2)時 D[1,3]還是∞，結果維持∞；稍後才算出 D[1,3]=2，卻不回訪(1,2)。正確距離是3。[S1 p695、699 ex25.2-4；S2 p17；S3 p17]

負對角線是警訊，−∞ 要按點對標記

完成 Floyd 後，存在 D[t,t]<0 當且僅當存在負環。負對角線代表一條負的封閉 walk，其中必有負圈；反向而言，負圈的某個頂點在允許全部中繼後會得到負對角線。

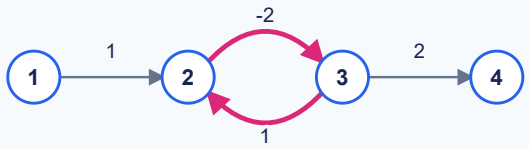
用可達矩陣 R 判定受影響點對。

$$\delta(i,j) = -\infty \iff \exists t:\ R[i,t] \wedge D[t,t] < 0 \wedge R[t,j].$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

負對角線是警訊，−∞ 要按點對標記 （續）

負環只影響可穿越它的點對



2→3→2 = −1

i ∈ {1, 2, 3}

j ∈ {2, 3, 4}

這 9 對距離為 −∞

δ(4,4)=0

δ(4,1)=+∞

粉紅邊標示本次重點路徑或負環；箭頭表示方向。

上圖是獨立反例，不是共用圖 G。只有 {1,2,3}×{2,3,4} 的9對受影響。先完成有限運算，再做標記；有負環時 Floyd 的有限輸出不是無限制距離。Johnson 選擇整體回報負環並停止。[S1 ex25.2-6、§25.3]

Johnson 用超級來源求每個頂點的位勢

新增 s，對每個 v 加入 s→v 權0。s 沒有入邊，因此不創造新環，也不改變原頂點間的路徑。每個原頂點都可從 s 到達，任何負環都會被 Bellman–Ford 偵測。

無負環才設 h(v)=δ(s,v)。共用圖的 h=(0,0,−2,0)，因 s→2→3 給 −2，其餘不低於0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford 各層求得位勢

同步 Bellman–Ford 的邊數層，順序為(s,1,2,3,4)。

最多邊數	距離向量
0	(0,∞,∞,∞,∞)
1	(0,0,0,0,0)
2	(0,0,0,−2,0)
3、4	(0,0,0,−2,0)

完成 |V′|−1=4 層後，第5層無改善才通過。實作若逐邊原地鬆弛，中間向量會受邊順序影響，最終位勢相同。[S1 p702–704；S2 p22–23；S3 p22–23]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短路三角不等式保證重加權非負

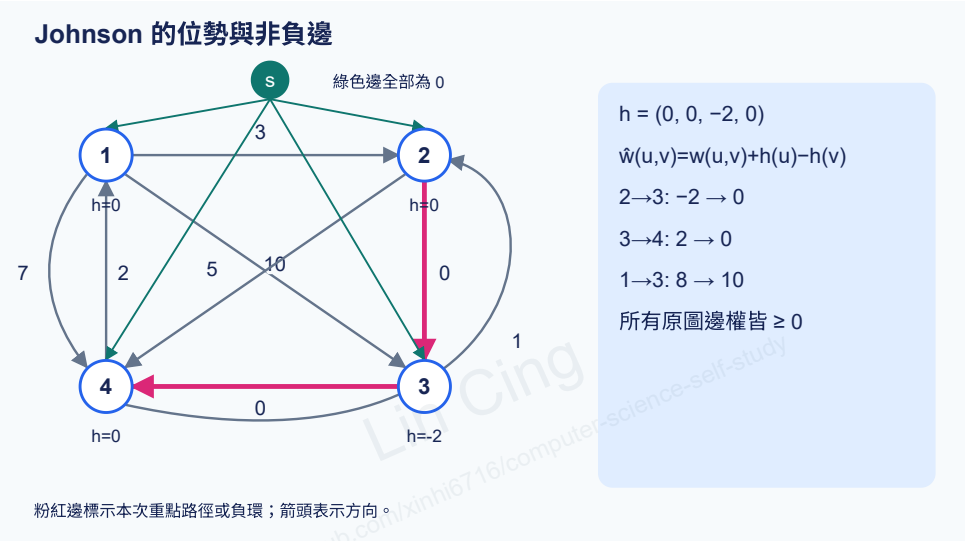
Bellman–Ford 的位勢滿足 h(v)≤h(u)+w(u,v)。移項即得

$$\widehat{w}(u,v) = w(u,v) + h(u) - h(v) \geq 0.$$

u→v	w	h(u)−h(v)	ŵ
1→2	3	0	3
1→3	8	2	10
1→4	7	0	7
2→3	-2	2	0
2→4	5	0	5
3→4	2	-2	0
4→1	2	0	2
4→2	1	0	1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短路三角不等式保證重加權非負 （續）



任意 h 都能保序，只有滿足差分限制的 h 才保證非負。每邊加同一常數無法替代位勢，因路徑邊數不同。[S1 lemma25.1、p702；S2 p19–23；S3 p19–23]

望遠鏡消去證明同端點路徑保序

對 $p=\langle v_0,\cdots,v_r\rangle$，

$$\widehat{w}(p)=\sum_{a=1}^{r}(w(v_{a-1},v_a)+h(v_{a-1})-h(v_a))=w(p)+h(v_0)-h(v_r).$$

內部位勢逐項相消。同一對端點的所有路徑都加上同一常數，所以大小關係與平手都保留，最短路集合不變。圈的端點相同，故圈權不變，也不可能把負環消成非負圈。

例如1→3直接權8，重加權10；1→2→3原權1，重加權3，兩者都加2。1→2→3→4與1→4的端點位勢差為0，權重仍為3與7。

因此 $\delta(u,v)=\hat{\delta}(u,v)-h(u)+h(v)$。先檢查可達，∞ 保持∞，不要對無窮值做有符號整數算術。[S1 lemma25.1；S2 p19–21；S3 p19–21]

每個來源跑 Dijkstra，再還原原距離

來源1在重加權圖依序定案1、2、3、4。抽出1後估計為(0,3,10,7)；抽出2後為(0,3,3,7)；抽出3後為(0,3,3,3)。

所有來源得到的重加權距離如下。

i \ j	1	2	3	4
1	0	3	3	3
2	2	0	0	0
3	2	1	0	0
4	2	1	1	0

每格套 $\delta=\delta-h(\text{來源})+h(\text{終點})$，還原成前面的 $D^{(4)}$。例如 $\delta(1,3)=3-0-2=1$，$\delta(3,1)=2-(-2)+0=4$。Dijkstra 的前驅路徑直接保留，因為保序證明已保證路徑相同。

檢核程式另以簡單路徑枚舉驗算全部16格，不以兩套 DP 一致當唯一證據。

圖密度與堆的選擇決定成本

方法	時間	適用條件
Slow min-plus	$\Theta(n^4)$	教學用邊數 DP
重複平方	$\Theta(n^3 \log n)$	教學用 min-plus 結合律
Floyd–Warshall	$\Theta(n^3)$	矩陣輸入、稠密圖
Johnson，binary heap	$O(n(m+n)\log n)$	稀疏鄰接串列
Johnson，Fibonacci heap	$O(nm+n^2\log n)$	攤銷 decrease-key 為 O(1)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較總成本與工作空間

顯式超級來源的 Bellman–Ford 為 $O(n(m+n))$，加上 n 次 Dijkstra 與 $\Theta(n^2)$ 還原。Fibonacci 版本稠密 $m=\Theta(n^2)$ 時 $O(n^3)$，稀疏 $m=O(n)$ 時 $O(n^2\log n)$；binary 版本稠密上界 $O(n^3\log n)$。完整輸出本來就需 $\Theta(n^2)$ 空間。

Floyd 工作空間 $\Theta(n^2)$，若保留全部教學層則 $\Theta(n^3)$。Johnson 除輸出外，鄰接串列與單來源工作空間 $O(m+n)$。

來源鍵 S0=原筆記 p18–19；S1=CLRS 第三版 §25.1–25.3，印刷 p684–705；S2=MIT 2004 lecture16 p1–23；S3=MIT 2005 lecture19 p1–23。完整路徑、讀取頁碼與內容對應見 sources.json。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapters 23–25. MIT 6.046J, Fall 2004, Lectures 13–16; Fall 2005, Lectures 16–19. Original handwritten algorithms notes (演算法_wjungle). These notes provide selected proof ideas and context; complete worked traces and diagrams are original educational synthesis.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study